Visual Programming

C# (C sharp)

Databases and LINQ

Lecturer: **Dr. Khaled Mohammed BinAbdl**
Department of Information Technology,
College of Computers, Seiyun University

Outline

- Introduction
- Relational databases
- LINQ to Entities and the ADO.NET Entity Framework
- Querying a database with LINQ

Introduction

- A **database** is an *organized* collection of data
- A database management system (**DBMS**) is a software system that provides mechanisms for storing, organizing, retrieving and modifying the data
- Most popular DBMSs today manage **relational databases**, which organize data simply as tables of rows and columns
- Popular proprietary DBMSs
  - Microsoft SQL Server, Oracle, Sybase, IBM DB2
- Popular open-source DBMSs
  - PostgreSQL, MySQL, MariaDB

Introduction (cont.)

- Here we use a version of Microsoft's SQL Server that's installed with Visual Studio
  - Called **SQL Server Express**, meant for development and testing of apps
  - Can also be downloaded separately from Microsoft's web site
  - Provides many features of Microsoft's full SQL Server product, but has some limitations, such as max database size of 10 GB
  - A SQL Server Express database file can be easily migrated to a full version of SQL Server

SQL, LINQ to Entities and the ADO.NET Entity Framework

- A language called **Structured Query Language (SQL)** is an international standard used with relational databases to perform **queries** (that is, to request information that satisfies given criteria) and to manipulate data
  - For years, programs that accessed a relational database passed SQL queries as strings to the database management system, then processed the results
- A more recent and more convenient way is to use an object-oriented query language called **LINQ to Entities**
  - the IDE provides *IntelliSense* for LINQ to Entities queries

SQL, LINQ to Entities and the ADO.NET Entity Framework

- The **ADO.NET Entity Framework** (or **EF**) enables apps to interact with data in various forms, including data stored in relational databases
- We'll use the ADO.NET EF and Visual Studio to create a so-called *entity data model* that represents the database, then use LINQ to Entities queries to access or manipulate objects in the entity data model
- The ADO.NET EF works with most popular database management systems, not just SQL Server
- Behind the scenes, the ADO.NET EF generates SQL statements that interact with a database

Relational Databases

- A relational database organizes data in tables
- Next slide illustrates a sample Employees table that might be used in a personnel system
- Tables are composed of rows (also called **records**) and columns (also called **fields**) in which values are stored
  - This table consists of six rows (one per employee) and five columns (one per attribute)
- The ID column of each row is the table's **primary key** - column (or group of columns) requiring a unique value
- LINQ to Entities requires *every table to have a primary key* to support updating the data in tables

ID	Name	Department	Salary	Location
23603	Jones	413	1100	New Jersey
24568	Kerwin	413	2000	New Jersey
34589	Larson	642	1800	Los Angeles
35761	Myers	611	1400	Orlando
47132	Neumann	413	9000	New Jersey
78321	Stephens	611	8500	Orlando

Row

Primary key

Column

Fig. 22.1 | `Employees` table sample data.

Relational Databases (cont.)

- Each column represents a different data attribute
- Some column values may be duplicated between rows
  - E.g., three different rows in the Employees table's Department column contain the number 413, indicating that these employees work in the same department
- You can use SQL, or LINQ to Entities, to define queries that select subsets of the data from a table
  - E.g., a program might select data from the Employees table to create a query result that shows where each department is located, in ascending order by Department number

Department	Location
413	New Jersey
611	Orlando
642	Los Angeles

Fig. 22.2 | Distinct `Department` and `Location` data from the `Employees` table.

A Books Database

- We now consider a simple Books database that stores information about some books
- This database has three tables
- A database's tables, their fields and the relationships among them are collectively known as a **database schema**
- The ADO.NET EF uses a database's schema to define classes that enable you to interact with the database
- SQL Server database files have the .mdf ("master data file") file-name extension
  - The database file (Books.mdf) is provided with this chapter's examples

Column	Description
`AuthorID`	Author's ID number in the database. In the `Books` database, this integer column is defined as an **identity** column, also known as an **autoincremented** column—for each row inserted in the table, the `AuthorID` value is increased by 1 automatically to ensure that each row has a unique `AuthorID`. This is the *primary key*.
`FirstName`	Author's first name (a string).
`LastName`	Author's last name (a string).

Fig. 22.3 | `Authors` table of the `Books` database.

AuthorID	FirstName	LastName
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Sue	Green
4	John	Purple

Fig. 22.4 | Data from the `Authors` table.

Column	Description
`ISBN`	ISBN of the book (a string). The table's primary key. ISBN is an abbreviation for "International Standard Book Number"—a numbering scheme that publishers worldwide use to give every book a *unique* identification number.
`Title`	Title of the book (a string).
`Edition-Number`	Edition number of the book (an integer).
`Copyright`	Copyright year of the book (a string).

Fig. 22.5 | `Titles` table of the `Books` database.

ISBN	Title	EditionNumber	Copyright
0132151006	Internet & World Wide Web How to Program	5	2012
0133807800	Java How to Program	10	2015
0132575655	Java How to Program, Late Objects Version	10	2015
0133976890	C How to Program	8	2016
0133406954	Visual Basic 2012 How to Program	6	2014
0134601548	Visual C# How to Program	6	2017

Fig. 22.6 | Data from the `Titles` table of the `Books` database. (Part 1 of 2.)

ISBN	Title	EditionNumber	Copyright
0134448235	C++ How to Program	10	2016
0134444302	Android How to Program	3	2016
0134289366	Android 6 for Programmers: An App-Driven Approach	3	2016
0133965260	iOS 8 for Programmers: An App-Driven Approach with Swift	3	2015
0134021363	Swift for Programmers	1	2015

Fig. 22.6 | Data from the `Titles` table of the `Books` database. (Part 2 of 2.)

AuthorISBN Table of the Books Database

- AuthorISBN table associates authors with their books
- The AuthorID column is a **foreign key** - a column in this table that matches the primary-key column in another table
  - The ISBN column is also a foreign key - it matches the primary-key column (that is, ISBN) in the Titles table
- A goal when designing a database is to minimize the amount of *duplicated* data among the database's tables
- Foreign keys, which are specified when a database table is created in the database, link the data in multiple tables
- Together the AuthorID and ISBN columns in this table form a *composite* primary key

Column	Description
`AuthorID`	The author's ID number, a foreign key to the `Authors` table.
`ISBN`	The ISBN for a book, a foreign key to the `Titles` table.

Fig. 22.7 | `AuthorISBN` table of the `Books` database.

AuthorID	ISBN	AuthorID	ISBN
1	0132151006	2	0132575655
1	0133807800	2	0133976890
1	0132575655	2	0133406954
1	0133976890	2	0134601548
1	0133406954	2	0134448235
1	0134601548	2	0134444302
1	0134448235	2	0134289366
1	0134444302	2	0133965260
1	0134289366	2	0134021363
1	0133965260	3	0132151006
1	0134021363	3	0133406954
2	0132151006	4	0134289366
2	0133807800	5	0134289366

Fig. 22.8 | Data from the `AuthorISBN` table of the `Books` database.

Entity-Relationship Diagram for the Books Database

- The next figure is an entity-relationship (ER) diagram for the Books database
- This diagram shows the tables in the database and the relationships among them
- The lines connecting the tables in represent the relationships among the tables
- The "1" represents the one end, while the "∞" represents the many end of a *one-to-many* relationship
- The relationships in the figure illustrate that the sole purpose of the AuthorISBN table is to provide a *many-to-many* relationship between the Authors and Titles tables—an author can write many books, and a book can have many authors

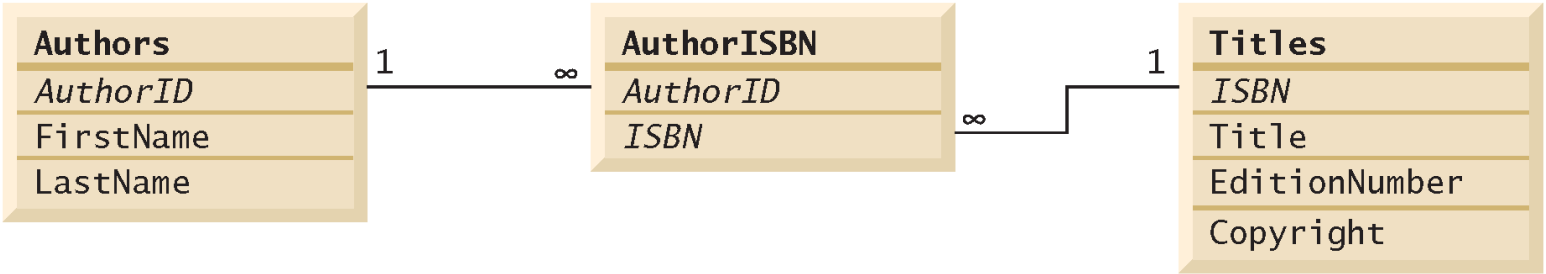

Fig. 22.9 | Entity-relationship diagram for the Books database.

LINQ to Entities and the ADO.NET Entity Framework

- When using the ADO.NET Entity Framework, you interact with the database via classes that the IDE generates from the database schema
- This process is initiated by adding a new ADO.NET Entity Data Model to your project
- For the Authors and Titles tables in the Books database, the IDE creates two classes each in the data model:
  - The first class represents *a row of the table* and contains properties for each column in the table
    - (row objects) store the data from individual rows of the table
  - The second class represents *the table itself*. An object of this class stores a collection of row objects that correspond to all of the rows in the table The table classes for the Books database are named Authors and Titles

LINQ to Entities and the ADO.NET Entity Framework (cont.)

- Relationships Between Tables in the Entity Data Model
  - Relationships between tables are taken into account in the entity data model's generated classes
  - E.g., the Author row class contains a navigation property named Titles, which can be used to get the Title objects that represent all the books written by that author
- DbContext Class
  - A DbContext manages the data flow between the program and the database
  - When the IDE generates the entity data model's row and table classes, it also creates a derived class of DbContext that is specific to the given database
  - For the Books database, this derived class has properties for the Authors and Titles tables. These can be used as data sources for manipulating data in LINQ queries and in GUIs. Any changes made to the data managed by the DbContext can be saved back to the database using the DbContext's SaveChanges method

Querying a Database with LINQ

- Here we demonstrate how to connect to a database, query it and display the results of the query
- There is little code - the IDE provides visual programming tools and wizards that simplify accessing data in apps
- These tools establish database connections and create the objects necessary to view and manipulate the data through Windows Forms GUI controls—a technique known as **data binding**
- We'll create one solution that contains several projects
  - One will be a reusable class library containing the ADO.NET Entity Data Model for interacting with the Books database
  - The other projects will be Windows Forms apps that use the ADO.NET EntityDataModel in the class library to manipulate the database

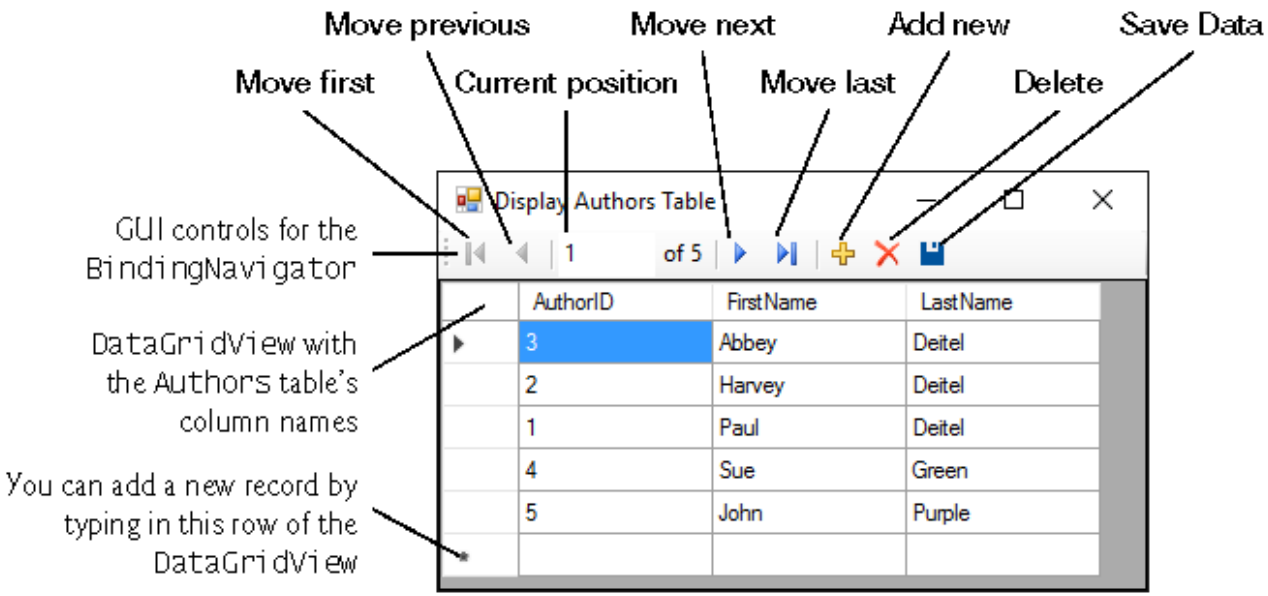

Fig. 22.10 | GUI for the **Display Authors Table** app.

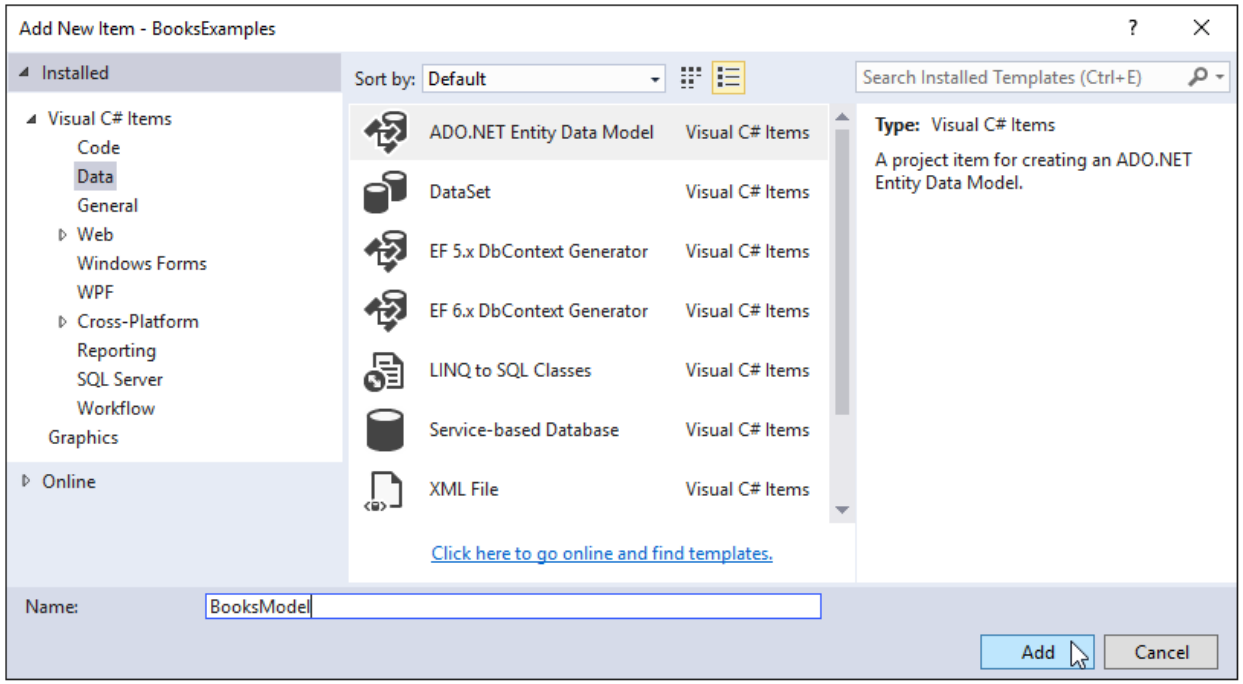

Fig. 22.11 | Selecting **ADO.NET Entity Data Model** in the **Add New Item** Dialog.

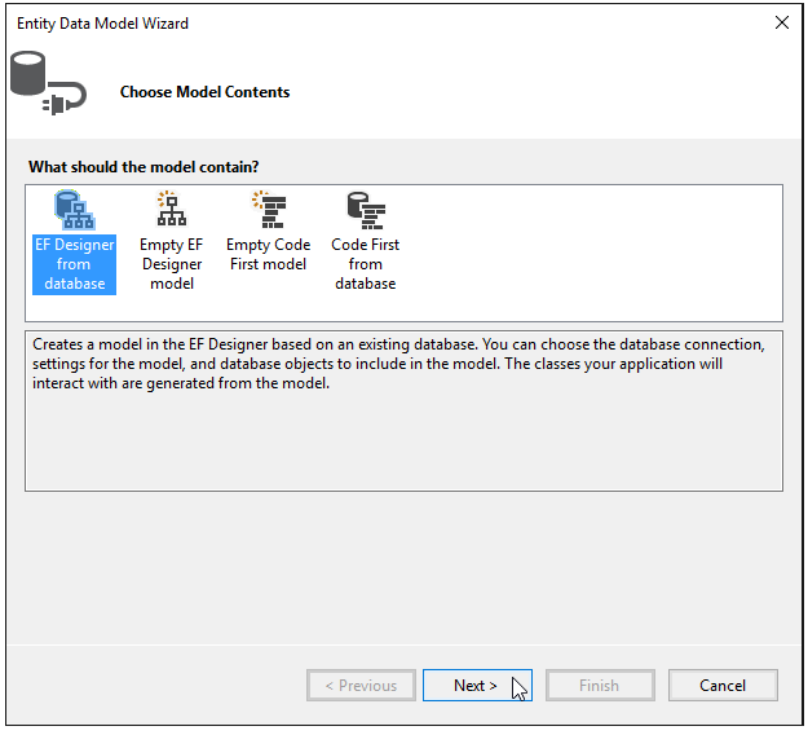

Fig. 22.12 | **Entity Data Model Wizard** dialog's **Choose Model Contents** step.

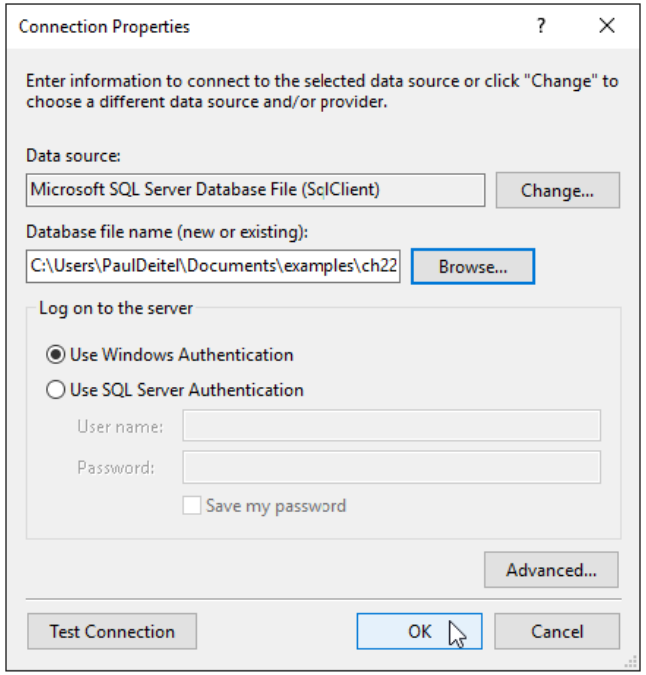

Fig. 22.13 | **Connection Properties** dialog.

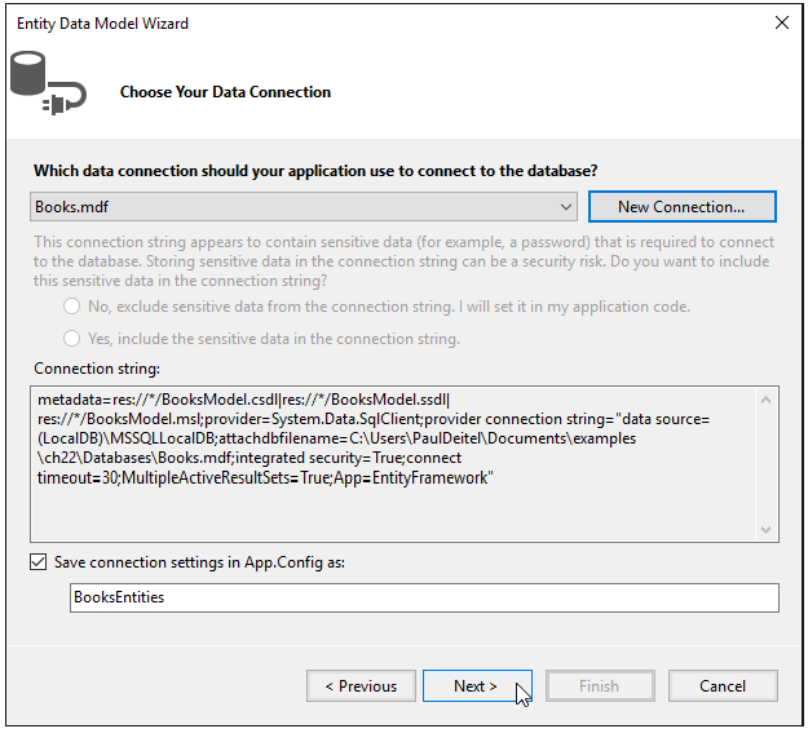

Fig. 22.14 | **Choose Your Data Connection** step after selecting `Books.mdf`.

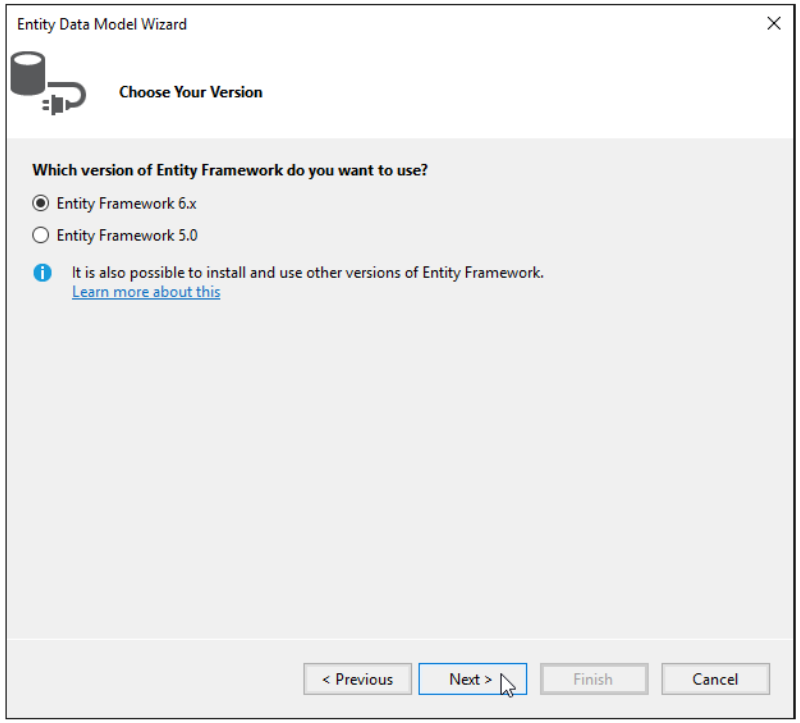

Fig. 22.15 | Choosing the Entity Framework version to use in the project.

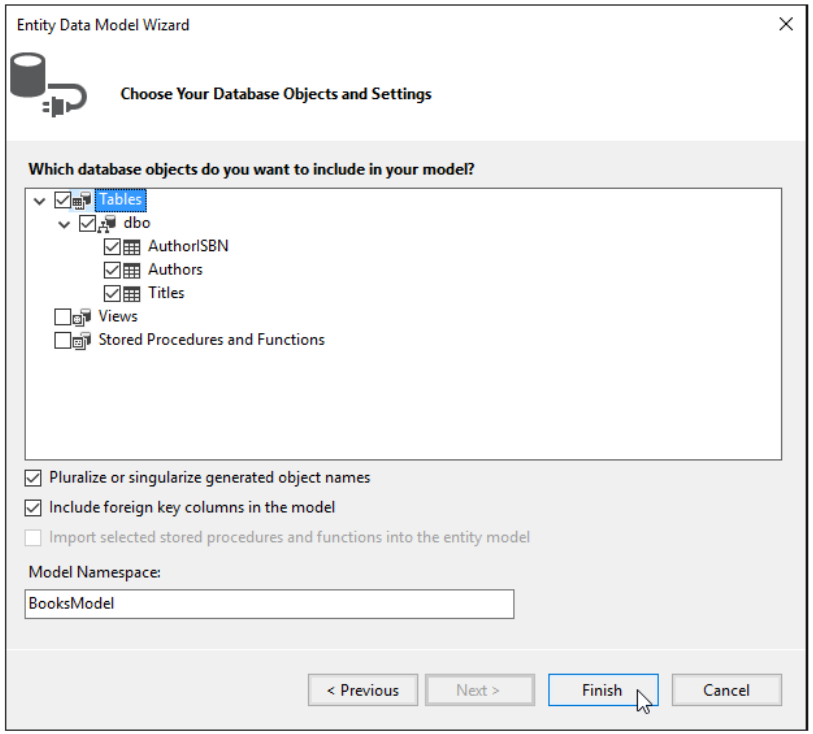

Fig. 22.16 | Selecting the database's tables to include in the ADO.NET Entity Data Model.

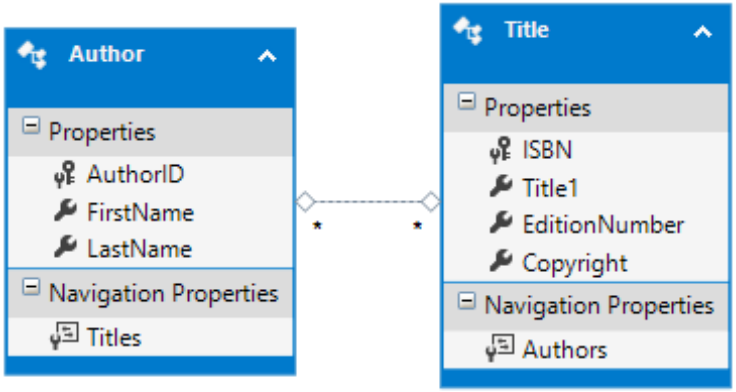

Fig. 22.17 | Entity data model diagram for the **Author** and **Title** entities.

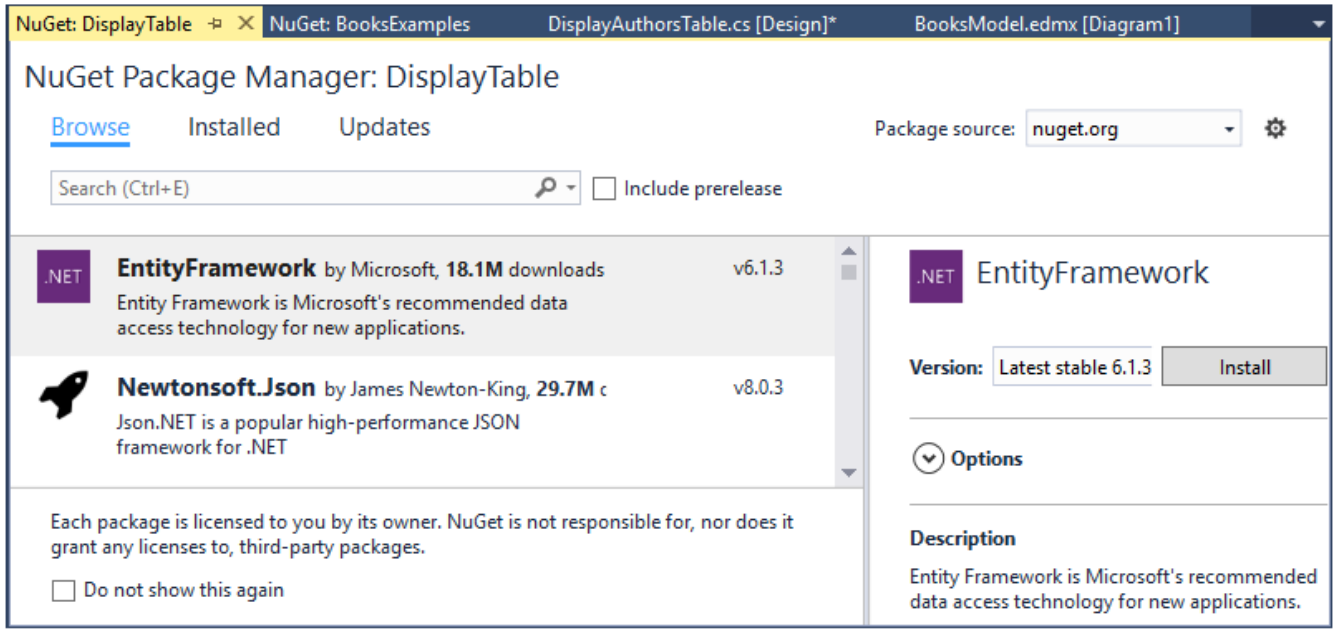

Fig. 22.18 | Selecting and installing the **EntityFramework** in the **NuGet** tab.

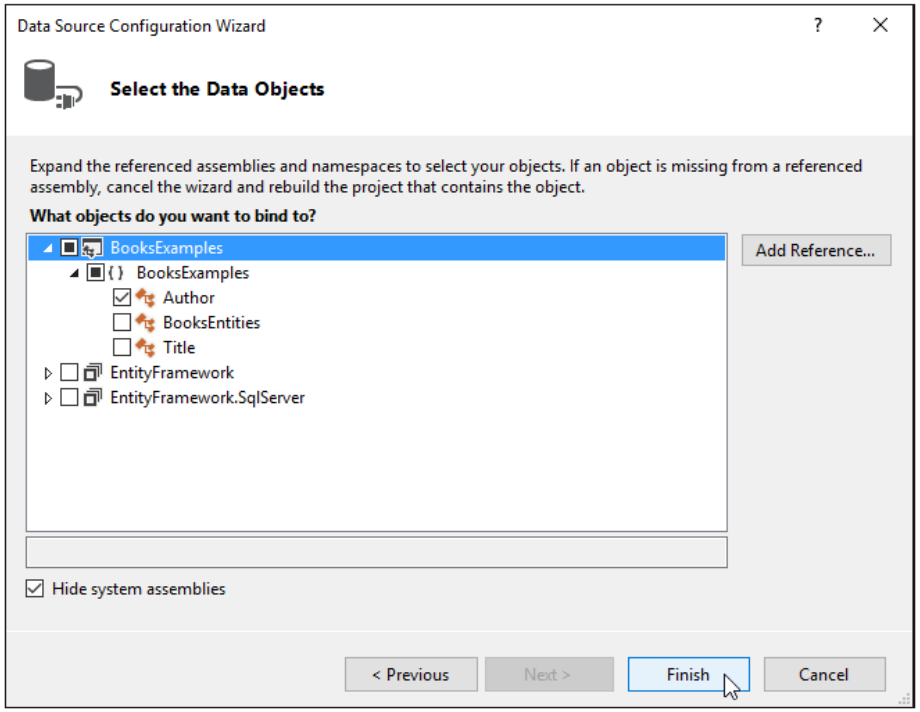

Fig. 22.19 | Selecting the Entity Data Model class `Author` as the data source.

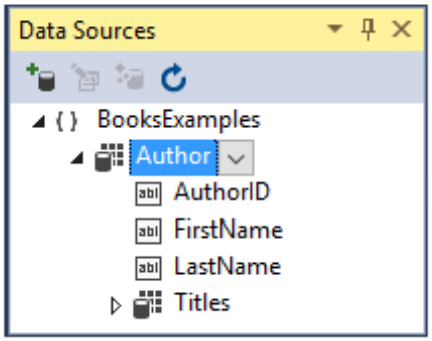

Fig. 22.20 | **Data Sources** window showing the expanded **`Author`** class as a data source.

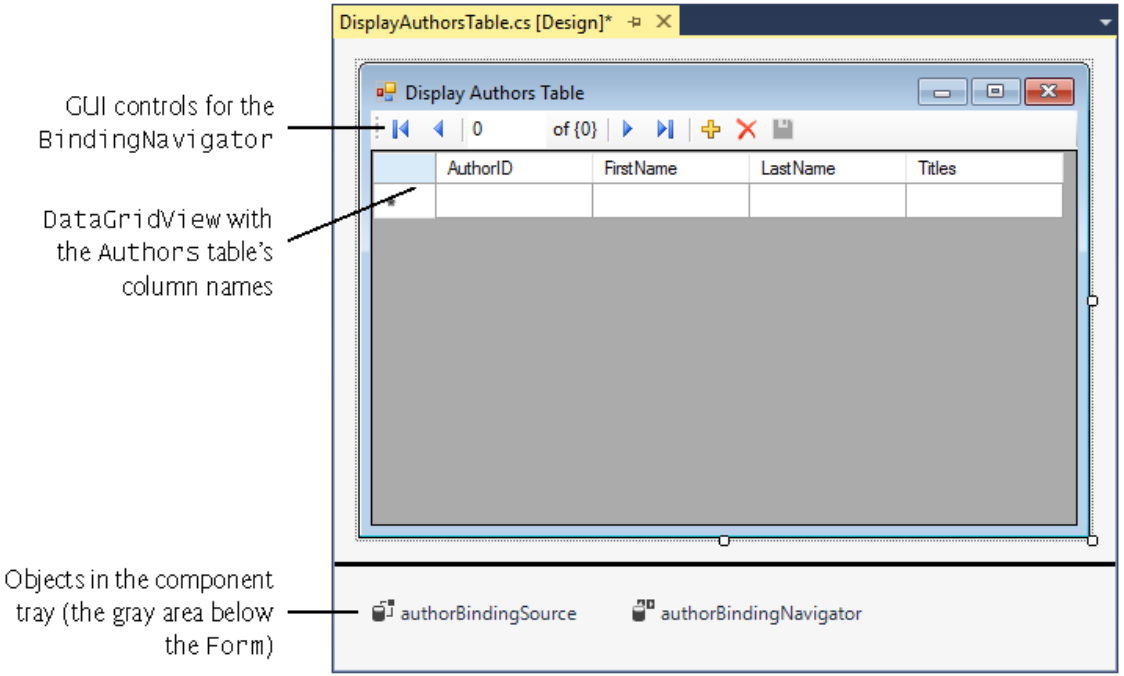

Fig. 22.21 | Component tray holds nonvisual components in **Design** view.

Connecting the Data Source to the authorBindingSource

- The final step is to connect the data source to the authorBindingSource, so that the app can interact with the database
- Next listing shows the code needed to *obtain* data from the database and to *save* any changes that the user makes to the data back into the database

```
// Fig. 22.22: DisplayAuthorsTable.cs
// Displaying data from a database table in a DataGridView.
using System;
using System.Data.Entity;
using System.Data.Entity.Validation;
using System.Linq;
using System.Windows.Forms;

namespace DisplayTable
{
   public partial class DisplayAuthorsTable : Form
   {
      // constructor
      public DisplayAuthorsTable()
      {
         InitializeComponent();
      }

```

Fig. 22.22 | Displaying data from a database table in a `DataGridView`. (Part 1 of 4.)

```
        // Entity Framework DbContext
        private BooksExamples.BooksEntities dbcontext =
           new BooksExamples.BooksEntities();

        // load data from database into DataGridView
        private void DisplayAuthorsTable_Load(object sender, EventArgs e)
        {
           // load Authors table ordered by LastName then FirstName
           dbcontext.Authors
              .OrderBy(author => author.LastName)
              .ThenBy(author => author.FirstName)
              .Load();

           // specify DataSource for authorBindingSource
           authorBindingSource.DataSource = dbcontext.Authors.Local;
        }
```

Fig. 22.22 | Displaying data from a database table in a `DataGridView`. (Part 2 of 4.)

```

        // click event handler for the Save Button in the
        // BindingNavigator saves the changes made to the data
        private void authorBindingNavigatorSaveItem_Click(
           object sender, EventArgs e)
        {
           Validate(); // validate the input fields
           authorBindingSource.EndEdit(); // complete current edit, if any

           // try to save changes
           try
           {
              dbcontext.SaveChanges(); // write changes to database file
           }
           catch(DbEntityValidationException)
           {
              MessageBox.Show("FirstName and LastName must contain values",
                 "Entity Validation Exception");
           }
        }
     }
  }
```

Fig. 22.22 | Displaying data from a database table in a `DataGridView`. (Part 3 of 4.)

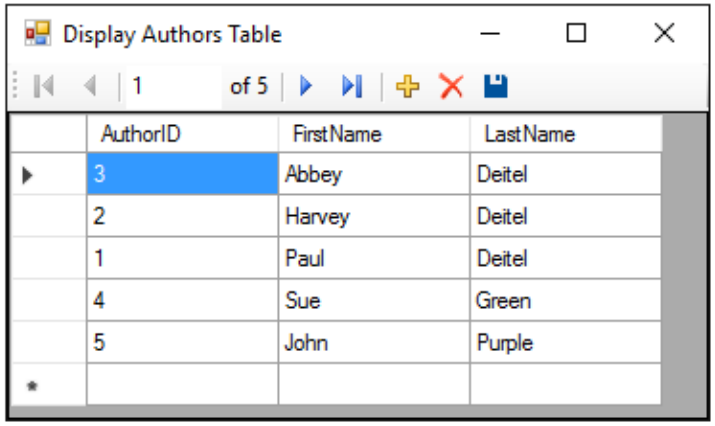

Fig. 22.22 | Displaying data from a database table in a **`DataGridView`**. (Part 4 of 4.)

DisplayAuthorsTable_Load Event Handler

- Double clicking the Form's title bar in Design view takes us to the Form's Load event handler
- In this app, we allow data to move between the DbContext and the database by using LINQ to Entities *extension methods* to extract data from the BooksEntities's Authors property, which corresponds to the Authors table in the database
  - *Extension methods in C# allow enhancing a class's capabilities without modifying its class's definition*

DisplayAuthorsTable_Load Event Handler

- The expression `dbContext.Authors` indicates that we wish to get data from the Authors table
- The OrderBy extension method call: `.OrderBy( author => author.LastName )`
  - indicates that the rows of the table should be retrieved in ascending order by the authors' last names
  - The argument to OrderBy is a **lambda expression** that defines a simple, **anonymous method**
  - A lambda expression begins with a parameter list - author in this case, which is an object of the Author entity data model class
  - The parameter list is followed by the => lambda operator (read as "goes to") and an expression that represents the body of the function
  - The value produced by the expression - a given author's last name - is *implicitly* returned by the lambda expression

LINQ to Entities

- LINQ to Entities has two variants:
  - method-based syntax, e.g.:

```
var numQuery2 = numbers.Where(num => num % 2 == 0).OrderBy(n => n);
```

  - query syntax, e.g.:

```
var numQuery1 =
  from num in numbers
  where num % 2 == 0
  orderby num
  select num;
```

- More information:
  - "*Query Syntax and Method Syntax in LINQ (C#)*", https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/query-syntax-and-method-syntax-in-linq

DisplayAuthorsTable_Load Event Handler

- Line 30 calls the **Load extension method** (defined in class **DBExtensions** from the namespace System.Data.Entity).
  - This method executes the LINQ to Entities query and loads the results into memory.
  - This data is tracked by the BookEntities DbContext in local memory so that any changes made to the data can eventually be saved into the database.
- Previous LINQ to Entities method based syntax is equivalent to the following query syntax:

```
(from author in dbcontext.Authors orderby author.LastName,
author.FirstName select author).Load();
```

- Line 33 sets the authorBindingSource's **DataSource property** to the Local property of the dbcontext.Authors object. In this case, the Local property is an ObservableCollection<Author> that represents the query results that were loaded into memory by the previous lines

Saving Modifications to the Database

- Saving the data entered in the DataGridView back to the database is a three-step process.
- First, all controls on the form are validated (line 41) by calling the DisplayTableForm's inherited Validate method—if any control has an event handler for the Validating event, it executes
- Next, line 42 calls **EndEdit** on the authorBindingSource, which forces it to save any pending changes into the BooksEntities model *in memory*
- Finally, line 47 calls SaveChanges on the BooksEntities object (dbcontext) to store any changes into the database